

TMC-104 / TMD-105 – ADVANCED OPERATING SYSTEMS

MCA / MCA (AI&DS) First Semester — Term Evaluation Theory (Odd) (Late/Lateral Admission) Exam,
Nov. 2025

Graphic Era Hill University | Complete Solved Answer Key with Detailed Explanations

Reference: *Operating System Concepts*, 9th Edition — Abraham Silberschatz, Peter B. Galvin, Greg Gagne

Note: The original paper requires attempting **one sub-question [(a) OR (b)]** from each of the 5 questions, 10 marks each (Total = 50 marks). For complete revision of the syllabus, **both alternatives of every question are solved in full below**. All comparison tables use wrapped text cells so content stays fully readable and never overlaps.

Q1. (a) Define an operating system. Also explain the layered architecture of an operating system.

[CO1] Marks: 10

Definition

An Operating System (OS) is system software that acts as an intermediary between the user/application programs and the computer hardware. It manages hardware resources such as the CPU, memory and I/O devices, and provides a convenient, efficient and secure environment for executing programs. Silberschatz describes it as both a "resource allocator", managing and allocating resources among competing requests, and a "control program" that controls execution of user programs to prevent errors and improper use of the system.

Layered Approach (Layered Architecture)

The layered approach structures the OS into a number of layers (levels), each built on top of the ones below it. The bottom layer (layer 0) is the hardware; the topmost layer (layer N) is the user interface. Each layer uses only the functions/services of the layers strictly below it and hides its own implementation from the layers above – a classic example of information hiding and modularity, first demonstrated in Dijkstra's THE operating system.

Layered Architecture of an Operating System (THE OS model)

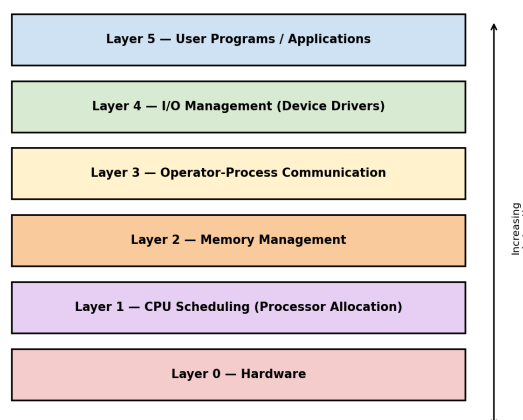


Fig. 1: Layered Architecture of an Operating System

Typical layer breakdown (illustrative, as in the THE OS)

- **Layer 0 – Hardware:** the physical CPU, memory and I/O devices.

- **Layer 1 – CPU Scheduling:** allocates the processor among competing processes.
- **Layer 2 – Memory Management:** allocates space for processes in main memory.
- **Layer 3 – Operator-Process Communication:** handles console/operator messages to/from processes.
- **Layer 4 – I/O Management:** buffers information transferred to/from peripheral devices via device drivers.
- **Layer 5 – User Programs / Applications:** user processes needing no knowledge of lower-level details.

Advantages: simplifies debugging/verification (each layer tested using only already-verified lower layers) and improves modularity (a layer's internals can change without affecting others, provided the interface is preserved).

Disadvantages: layers must be carefully defined since a layer can call only lower layers (restrictive for functions like I/O that may be needed at multiple layers), and each layer adds call overhead, making a layered system generally less efficient than a monolithic one.

Q1. (b) Calculate Average waiting time, Average TAT, throughput, CPU utilization using FCFS for the following:

[CO1] Marks: 10

Process	Arrival Time	CPU Burst Time (ms)
P0	3	2
P1	2	4
P2	0	6
P3	1	4

Concept: First-Come-First-Served (FCFS) is a non-preemptive algorithm where processes are executed strictly in order of arrival.

Step 1: Determine execution order by arrival time

- P2 arrives first at $t=0 \rightarrow$ runs 0–6.
- P3 arrives at $t=1$ (waits till 6) $\rightarrow$ runs 6–10.
- P1 arrives at $t=2$ (waits till 10) $\rightarrow$ runs 10–14.
- P0 arrives at $t=3$ (waits till 14) $\rightarrow$ runs 14–16.

Gantt Chart — FCFS Scheduling

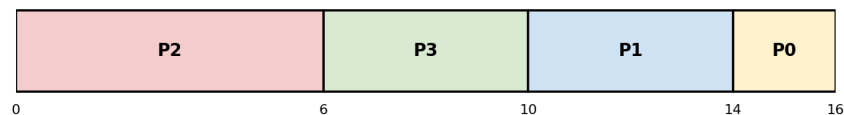


Fig. 2: Gantt chart for FCFS scheduling

Step 2: Completion, Turnaround (TAT) and Waiting Time

TAT = Completion Time – Arrival Time | WT = Turnaround Time – Burst Time

Process	Arrival	Burst	Completion	Turnaround (CT-AT)	Waiting (TAT-BT)
P2	0	6	6	$6-0 = 6$	$6-6 = 0$
P3	1	4	10	$10-1 = 9$	$9-4 = 5$
P1	2	4	14	$14-2 = 12$	$12-4 = 8$
P0	3	2	16	$16-3 = 13$	$13-2 = 11$

Step 3: Averages

- Average Turnaround Time = $(6+9+12+13)/4 = 40/4 = 10$ ms
- Average Waiting Time = $(0+5+8+11)/4 = 24/4 = 6$ ms

Step 4: CPU Utilization

Total elapsed time = 16 ms; total burst time = $6+4+4+2 = 16$ ms. Since P2 starts exactly at $t=0$ and the CPU is never idle thereafter, CPU Utilization = $(16/16) \times 100 = 100\%$.

Step 5: Throughput

Throughput = Number of processes completed / Total time = $4/16 = 0.25$ processes per ms (1 process every 4 ms).

Q2. (a) Explain the various file systems used in different operating systems.

[CO1] Marks: 10

A file system defines how data is named, stored, organized and retrieved on a storage device. Different operating systems use different file systems, optimized for their design goals:

OS	File System(s)	Key Feature
Windows	FAT32, exFAT, NTFS	NTFS supports journaling (crash recovery), permissions via ACLs, compression, encryption (EFS) and large file/volume sizes; FAT32/exFAT are simpler, used mainly for removable media.
Linux/UNIX	ext2, ext3, ext4, XFS, Btrfs	ext3/ext4 add journaling for reliability; use inodes and block groups; Btrfs adds copy-on-write, snapshots and checksums.
macOS	APFS (formerly HFS+)	APFS is optimized for flash/SSD storage, supports snapshots, cloning and strong native encryption.
Optical/removable media	ISO 9660, UDF	Designed for read-only or write-once optical media such as CDs/DVDs.
Network file systems	NFS, SMB/CIFS	Allow files on a remote server to be accessed as if they were local, over a network.

Regardless of the underlying on-disk format, every file system must provide four core capabilities (Silberschatz, Ch.11–12):

- **Naming** – a convention for identifying files (names, extensions, case sensitivity rules).
- **Directory structure** – organizing files hierarchically (single-level, two-level, tree-structured, or acyclic-graph directories) so users can group related files logically.
- **Space allocation** – deciding how disk blocks are assigned to a file: *contiguous allocation* (fast sequential access, but suffers external fragmentation), *linked allocation* (no external fragmentation, but slow direct access and pointer overhead), or *indexed allocation* (an index block holds all block pointers, giving good direct access without the external fragmentation of contiguous allocation – used by ext2/ext3/ext4 inodes and NTFS's Master File Table).
- **Free-space management** – tracking unused blocks via a bit vector, a linked free list, grouping, or a counting method, so space can be allocated to new files.

Thus, while Windows (NTFS), Linux (ext4) and macOS (APFS) differ in their exact on-disk data structures, they all solve the same underlying problems of naming, organizing, allocating space to, and protecting files.

Q2. (b) Write a short note on Access Control. Also explain the access permissions available in LINUX.

[CO1] Marks: 10

Access Control

Access control is the security mechanism by which an operating system regulates who (which users/processes) can view, use or modify which resources (files, devices, memory) and what operations they are permitted to perform on them. It is central to enforcing both confidentiality and integrity. Common access control models include:

- **Discretionary Access Control (DAC)** – the owner of a resource decides who else can access it and with what permissions (this is the model used by standard UNIX/Linux file permissions).
- **Mandatory Access Control (MAC)** – access is governed by a central, system-enforced security policy (e.g., security labels/clearances) that individual users cannot override – used in SELinux, AppArmor.
- **Role-Based Access Control (RBAC)** – permissions are assigned to roles (e.g., 'admin', 'auditor') rather than individual users, and users are granted access by being assigned to appropriate roles.

The theoretical foundation for all these models is the **access matrix**, where rows represent protection domains and columns represent objects, with each cell listing the operations permitted – access control lists (per-object) and capability lists (per-domain) are two practical ways of implementing this matrix efficiently.

Access Permissions in LINUX

Linux controls access to every file and directory using permissions defined for three user classes and three permission types.

Class	Meaning	
Owner (u)	The user who owns the file.	
Group (g)	Users belonging to the file's group.	
Others (o)	All other users on the system.	

Permission	On a File	On a Directory
Read (r)	View file contents.	List the files inside the directory.
Write (w)	Modify/overwrite file contents.	Create, delete or rename files within the directory.
Execute (x)	Run the file as a program/script.	Enter (cd) the directory and access files inside it by name.

Permissions are shown as a 10-character string, e.g. **-rwxr-xr--**: the leading character denotes file type (- for regular file, d for directory), followed by three permission triplets for owner, group and others respectively. They can also be expressed numerically (octal, r=4, w=2, x=1 summed per class) – e.g. **chmod 754 file** sets rwx for owner, r-x for group and r-- for others. The commands *chmod* (change mode), *chown* (change owner) and *chgrp* (change group) are used to manage these permissions.

Q3. (a) What do you understand by kernel of an OS? Also explain the different types of kernels.

[CO1] Marks: 10

Kernel of an OS

The kernel is the core component of an operating system – the part loaded into memory first and kept resident throughout system operation. It runs in privileged (kernel) mode with full access to the hardware, and is responsible for the most fundamental functions: process and memory management, handling interrupts and system calls, and controlling communication with I/O devices. All other software, including the shell and applications, depends on services provided by the kernel.

Types of Kernels

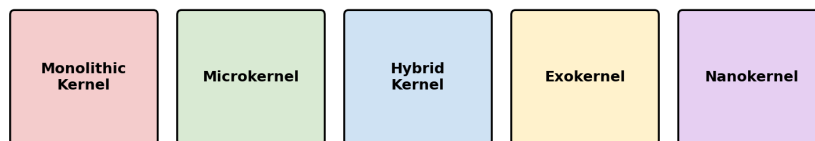


Fig. 3: Types of Kernels

1. Monolithic Kernel

The entire OS (process management, memory management, file systems, device drivers) runs as one large program in kernel space. Components communicate via direct function calls, which is fast, but a bug in any part can crash the whole system. **Example:** traditional UNIX, Linux, MS-DOS.

2. Microkernel

Only the most essential services (IPC, minimal scheduling, minimal memory management) run in kernel mode; other services (file systems, device drivers) run as separate user-space servers communicating via message passing. More robust/modular, but message passing adds overhead. **Example:** Mach, QNX, MINIX.

3. Hybrid Kernel

Combines features of monolithic and microkernels: keeps a small set of core services in kernel space for modularity like a microkernel, but also runs some additional services in kernel space for performance, unlike a pure microkernel. **Example:** Windows NT/10/11 kernel, macOS XNU kernel.

4. Exokernel

A minimal kernel that securely allocates raw hardware resources (CPU cycles, disk blocks, memory pages) to library operating systems running in user space, without imposing any fixed abstraction, giving applications maximum control and efficiency. **Example:** MIT Exokernel, Nemesis.

5. Nanokernel

An extremely minimal kernel providing only the most basic hardware abstraction (often just interrupt handling), delegating virtually everything else, including device drivers, to higher-level software. **Example:** KeyKOS, the L4 microkernel family (in its nanokernel form).

Q3. (b) Explain the different types of attributes and operations related to files.

[CO1] Marks: 10

File Attributes

A file is named for the convenience of human users, and is referred to by that name. Besides a name, a file has other attributes that vary between operating systems, but typically include (Silberschatz, Ch.11):

- **Name** – the symbolic file name, the only information kept in human-readable form.
- **Identifier** – a unique tag (usually numeric) identifying the file within the file system.
- **Type** – needed for systems that support different file types (e.g., .txt, .exe, .jpg).
- **Location** – a pointer to the device and to the location of the file on that device.
- **Size** – the current size (in bytes/words/blocks), and possibly the maximum allowed size.
- **Protection** – access-control information determining who can read, write or execute the file.
- **Time, date, and user identification** – data kept for creation, last modification and last use, useful for protection, security and usage monitoring.

This information is stored in the directory structure/inode on secondary storage.

File Operations

A file is an abstract data type, and the OS provides several basic operations to manipulate it:

- **Creating a file** – space is found in the file system and a new directory entry is made.
- **Writing a file** – a system call specifying the file and the data to write; the system keeps a write pointer to the next write location.
- **Reading a file** – a system call specifying the file and destination; the system keeps a read pointer to the next read location.
- **Repositioning within a file (seek)** – the current-file-position pointer is repositioned without any actual I/O taking place.
- **Deleting a file** – the directory entry is located and the associated file space is released.
- **Truncating a file** – contents are erased but attributes are kept, resetting length to zero without deleting/recreating the file.
- **Opening/Closing a file** – open() brings a file's attributes and block locations into main memory for fast subsequent access; close() releases these resources and flushes any buffered writes to disk.

Q4. (a) Write a short note on (i) SVR3 (ii) Domain of protection.

[CO2] Marks: 10 (5 marks each)

(i) SVR3 (UNIX System V Release 3)

UNIX System V Release 3 (SVR3) was a major commercial UNIX release by AT&T (1986), building on the earlier System V lineage that established UNIX as an industry-standard OS. Key contributions of SVR3 include:

- **STREAMS** – a flexible, modular framework for implementing character-based I/O, especially networking protocol stacks, allowing protocol modules to be pushed/popped dynamically onto a communication channel.
- **Remote File Sharing (RFS)** – a distributed file system protocol allowing files on a remote UNIX system to be mounted and accessed as if local (a UNIX-specific precursor/alternative to NFS).
- **Shared libraries** – support for dynamically linked libraries, reducing the memory footprint and disk space needed by executables that use common library code.
- **Enhanced inter-process communication (IPC)** – refined System V IPC mechanisms (shared memory, semaphores, message queues) used for efficient communication between cooperating processes.

SVR3 (and the subsequent SVR4, which merged features from BSD UNIX and SunOS) heavily influenced the design of later commercial UNIX variants and, indirectly, of Linux.

(ii) Domain of Protection

A computer system is a collection of processes and objects (both hardware, such as CPU, memory, I/O devices, and software, such as files, programs, semaphores). A **protection domain** specifies the set of resources (objects) that a process may access, and the operations (rights) it is allowed to perform on each – formally, a domain is a set of (object, rights-set) pairs. Every process executes within some protection domain at any given time; the same object may be accessible from multiple domains, each possibly with a different set of allowed operations.

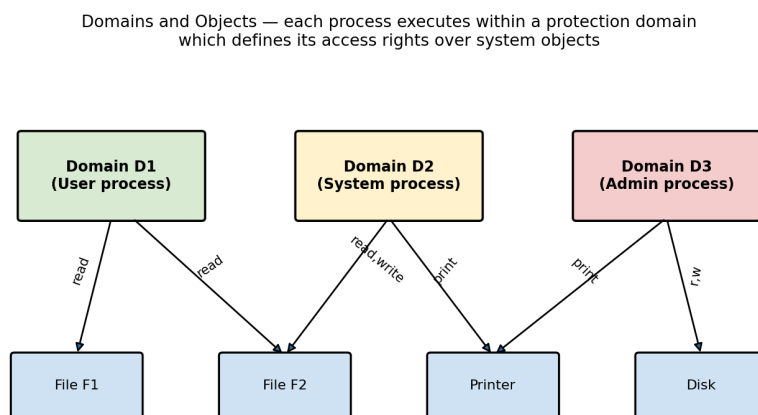


Fig. 4: Domains and the objects/rights they are permitted to access

- **Static association** – the set of resources available to a process is fixed for its lifetime (simple, but inflexible, and violates the need-to-know principle over time).
- **Dynamic association** – a process can switch from one domain to another during execution, e.g., via a system call or trap (as in UNIX user → kernel mode switching, or a setuid program temporarily acquiring the owner's rights).
- In UNIX, a domain can be associated directly with a user; the set of objects accessible depends on the identity of the user (and group). The **setuid** bit allows a domain switch: while executing a setuid program, the user's domain temporarily includes the file owner's access rights.

The domain-of-protection concept underlies the broader **access matrix** model, where each row is a domain, each column is an object, and each entry lists the operations permitted – the OS must track which domain a process is currently in and enforce only the rights defined for that domain.

Q4. (b) Write a short note on (i) Non-preemptive (ii) Preemptive scheduling.

[CO2] Marks: 10 (5 marks each)

(i) Non-preemptive Scheduling

In non-preemptive scheduling, once the CPU is allocated to a process, that process keeps the CPU until it either voluntarily releases it (by terminating) or switches to the waiting state (e.g., to request I/O). The scheduler cannot forcibly take the CPU away from a running process. Decisions to schedule occur only when a process terminates or blocks. **Examples:** First-Come-First-Served (FCFS), non-preemptive Shortest Job First (SJF), non-preemptive Priority scheduling.

(ii) Preemptive Scheduling

In preemptive scheduling, the OS can forcibly suspend (preempt) a running process – for example, when a higher-priority process becomes ready, or when the running process's allotted time quantum expires – and reassign the CPU to another process. The preempted process is moved back to the ready queue to resume later. **Examples:** Round Robin (RR), preemptive Priority scheduling, preemptive SJF (Shortest Remaining Time First).

Comparison

Basis	Non-preemptive	Preemptive
CPU can be taken away?	No – only released voluntarily by the process.	Yes – the OS can forcibly interrupt and reassign the CPU.
Overhead	Lower – fewer context switches.	Higher – frequent context switches consume CPU cycles.
Response time for high priority tasks	Can be poor – a long-running low-priority process can block urgent tasks.	Better – high-priority/urgent tasks can interrupt lower-priority ones promptly.
Starvation risk	Lower-priority processes are not interrupted, but a long process can still delay others behind it.	Lower-priority processes can be starved if higher-priority processes keep arriving.
Suitability	Simple batch systems where fairness/urgency is not critical.	Interactive/real-time systems needing fast, predictable response.
Example algorithms	FCFS, non-preemptive SJF, non-preemptive Priority.	Round Robin, preemptive SJF (SRTF), preemptive Priority.

Q5. (a) Consider the following set of processes with their arrival and burst times. Use Round Robin Scheduling with a Time Quantum = 2 units to calculate: Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT). Draw the Gantt Chart.

[CO2] Marks: 10

Process	Arrival Time	Burst Time
P1	0	5
P2	1	4
P3	2	2
P4	3	1

Concept: Round Robin (RR) is a preemptive scheduling algorithm designed for time-sharing systems. Each process gets the CPU for a fixed time quantum (here, 2 units); if it does not finish within that quantum, it is preempted and placed at the back of the ready queue. If a new process arrives at the exact same instant another process is preempted, the convention followed below is: the newly arrived process is placed in the ready queue before the just-preempted process.

Step 1: Simulate the ready queue (Quantum = 2)

- t=0: Ready queue = [P1]. Run P1 for 2 units → (0–2). Remaining P1 = 3. (P2 arrives at t=1, enqueued.)
- t=2: P3 arrives exactly now, enqueued after P2 but before the preempted P1. Queue = [P2, P3, P1(3)]. Run P2 for 2 units → (2–4). Remaining P2 = 2. (P4 arrives at t=3, enqueued.)
- t=4: Queue = [P3, P1(3), P4, P2(2)]. Run P3 for 2 units → (4–6). P3's remaining burst was exactly 2, so P3 **completes** at t=6.
- t=6: Queue = [P1(3), P4, P2(2)]. Run P1 for 2 units → (6–8). Remaining P1 = 1, re-queued.
- t=8: Queue = [P4, P2(2), P1(1)]. Run P4 – its remaining burst is only 1, so it needs just 1 unit → (8–9). P4 **completes** at t=9.
- t=9: Queue = [P2(2), P1(1)]. Run P2 for 2 units → (9–11). P2's remaining burst was exactly 2, so P2 **completes** at t=11.
- t=11: Queue = [P1(1)]. Run P1 – remaining burst 1 → (11–12). P1 **completes** at t=12.

Gantt Chart — Round Robin Scheduling (Time Quantum = 2)

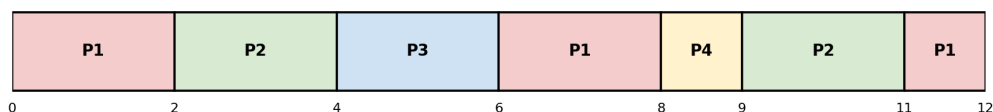


Fig. 5: Gantt chart for Round Robin scheduling (Time Quantum = 2)

Step 2: Completion, Turnaround and Waiting Time

TAT = Completion Time – Arrival Time | WT = Turnaround Time – Burst Time

Process	Arrival	Burst	Completion (CT)	Turnaround (TAT)	Waiting (WT)
P1	0	5	12	12-0 = 12	12-5 = 7
P2	1	4	11	11-1 = 10	10-4 = 6
P3	2	2	6	6-2 = 4	4-2 = 2
P4	3	1	9	9-3 = 6	6-1 = 5

Step 3: Averages

- Average Turnaround Time = $(12+10+4+6)/4 = 32/4 = \mathbf{8\ ms}$
- Average Waiting Time = $(7+6+2+5)/4 = 20/4 = \mathbf{5\ ms}$

Q5. (b) Differentiate between (i) Multiprogramming and Multitasking Operating Systems (ii) Kernel and Shell

[CO1] Marks: 10 (5 marks each)

(i) Multiprogramming OS vs Multitasking OS

Basis	Multiprogramming OS	Multitasking OS
Concept	Several jobs reside in main memory simultaneously so the CPU always has a job to execute whenever it becomes idle.	An extension of multiprogramming; the CPU rapidly switches between multiple tasks using small time-slices, so each task appears to run concurrently.
Primary Goal	Maximize CPU utilization by minimizing idle time.	Provide fast, interactive response time to the user.
Switching trigger	CPU switches to another job mainly when the current job requests I/O.	CPU switches based on a fixed time quantum, via preemptive (time-sharing) scheduling.
User interaction	Little or no direct interaction; jobs mostly run in the background.	Highly interactive; the user perceives simultaneous execution of multiple applications.
Example	Old mainframe batch-cum-multiprogrammed systems.	Modern desktop/laptop OS such as Windows or Linux running several apps together.

(ii) Kernel vs Shell

Basis	Kernel	Shell
Definition	The core component of the OS that directly manages hardware resources (CPU, memory, I/O).	A user-interface program that interprets and executes commands, acting as a bridge between the user and the kernel.
Mode of execution	Runs in privileged (kernel) mode with unrestricted hardware access.	Runs in user mode, and itself uses system calls to request kernel services.
Visibility to user	Not directly visible or accessible to the ordinary user.	Directly visible/interactive; the user types commands into it (e.g., a terminal prompt).
Replaceability	Cannot be replaced without effectively changing the OS itself (though modules can be loaded).	Can be changed independently, e.g., a user can switch from bash to zsh without changing the kernel.
Function	Handles process scheduling, memory management, device drivers, system calls.	Parses user commands (or scripts), launches programs, and displays output back to the user.
Example	Linux kernel, Windows NT kernel, XNU (macOS).	bash, zsh, csh (Linux/UNIX); Command Prompt, PowerShell (Windows).

Prepared for MCA / MCA (AI&DS) First Semester revision. Primary reference: Operating System Concepts, 9th Edition by Abraham Silberschatz, Peter B. Galvin and Greg Gagne (Wiley). Diagrams are original illustrations created for this answer key based on standard OS textbook concepts.